

lower_lr

April 17, 2026

```
[1]: # Cell 1: Install
!pip install -q catboost scikit-learn pandas numpy
```

[notice] A new release of pip is available: 26.0 -> 26.0.1
[notice] To update, run:
python3.14 -m pip install --upgrade pip

```
[2]: # Cell 2: Imports
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

from catboost import CatBoostClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import balanced_accuracy_score, classification_report

# Same 6 seeds that gave LB 0.37468
SEEDS = [42, 7, 123, 13, 99, 2024]
N_SPLITS = 10
print(f'Seeds: {SEEDS}')
```

Seeds: [42, 7, 123, 13, 99, 2024]

```
[3]: # Cell 3: Load data
TRAIN_DATA = pd.read_csv('train-data.csv', index_col='id')
TRAIN_LABEL = pd.read_csv('train-label.csv', index_col='id')
TEST_DATA = pd.read_csv('test-data.csv', index_col='id')

print(f'Train: {TRAIN_DATA.shape}, Test: {TEST_DATA.shape}')
```

Train: (13249, 41), Test: (8834, 41)

```
[4]: # Cell 4: Preprocessing (identical to 6-seed best run)
def preprocess(df):
    df = df.copy()
    drop_cols = [
```

```

        'first_name', 'last_name', 'insitute_name', 'institute_location',
        'test_1', 'test_2', 'test_3', 'test_4', 'test_5', 'treatment_consent'
    ]
    df = df.drop(columns=drop_cols)

    miss_cols = [
        'gender', 'maternal_defect', 'mother_age', 'father_age',
        'respiration', 'heart_rate', 'risk_level', 'place_birth',
        'folic_acid', 'maternal_illness', 'infertility_treatment',
        'problem_previous_pregnancies', 'abortion_cnt',
        'birth_defects', 'white_blood_cell_count', 'blood_test',
        'symptom_1', 'symptom_2', 'symptom_3', 'symptom_4', 'symptom_5'
    ]
    df['missing_count'] = df[miss_cols].isna().sum(axis=1)
    df['missing_parent_age'] = df['mother_age'].isna().astype(int) +
    df['father_age'].isna().astype(int)
    df['missing_symptoms'] =
    df[['symptom_1', 'symptom_2', 'symptom_3', 'symptom_4', 'symptom_5']].isna().
    sum(axis=1)
    df['missing_clinical'] =
    df[['respiration', 'heart_rate', 'risk_level', 'blood_test']].isna().sum(axis=1)

    for col in ['mother_age', 'father_age', 'maternal_defect', 'gender',
               'risk_level', 'heart_rate', 'respiration', 'abortion_cnt', 'white_blood_cell_count']:
        df[f'{col}_missing'] = df[col].isna().astype(int)

    binary_yn = [
        'mother_defect', 'father_defect', 'maternal_defect', 'paternal_defect',
        'alive', 'folic_acid', 'maternal_illness', 'infertility_treatment',
        'problem_previous_pregnancies',
        'symptom_1', 'symptom_2', 'symptom_3', 'symptom_4', 'symptom_5'
    ]
    for col in binary_yn:
        df[col] = df[col].map({'Y': 1, 'N': 0})

    df['respiration'] = df['respiration'].map({'A': 1, 'N': 0})
    df['heart_rate'] = df['heart_rate'].map({'A': 1, 'N': 0})
    df['risk_level'] = df['risk_level'].map({'H': 1, 'L': 0})
    df['place_birth'] = df['place_birth'].map({'I': 1, 'H': 0})
    df['birth_defects'] = df['birth_defects'].map({'S': 1, 'M': 2})
    df['gender'] = df['gender'].map({'M': 0, 'F': 1, 'A': 2})
    df['autopsy'] = df['autopsy'].map({'Y': 1, 'N': 0})
    for col in ['birth_asphyxia', 'radiation_exposure', 'substance_abuse']:
        df[col] = df[col].map({'Y': 1, 'N': 0, 'NR': 2})
    df['blood_test'] = df['blood_test'].map({'N': 0, 'I': 1, 'S': 2, 'A': 3})

```

```

    df['defect_sum'] =
↳df[['mother_defect', 'father_defect', 'maternal_defect', 'paternal_defect']].
↳sum(axis=1)
    df['symptom_sum'] =
↳df[['symptom_1', 'symptom_2', 'symptom_3', 'symptom_4', 'symptom_5']].sum(axis=1)
    df['defect_x_symptom'] = df['defect_sum'] * df['symptom_sum']
    df['any_defect'] = (df['defect_sum'] > 0).astype(int)
    df['any_symptom'] = (df['symptom_sum'] > 0).astype(int)
    df['high_symptom'] = (df['symptom_sum'] >= 4).astype(int)
    df['all_defects'] = (df['defect_sum'] == 4).astype(int)
    df['parent_age_gap'] = (df['father_age'] - df['mother_age']).abs()
    df['symptom_defect_ratio'] = df['symptom_sum'] / (df['defect_sum'] + 1)
    df['s4_and_s5'] = ((df['symptom_4'] == 1) & (df['symptom_5'] == 1)).
↳astype(int)
    df['no_s4_s5'] = ((df['symptom_4'] == 0) & (df['symptom_5'] == 0)).
↳astype(int)
    df['late_vs_early'] = (df['symptom_4'].fillna(0) + df['symptom_5'].fillna(0)
        - df['symptom_1'].fillna(0) - df['symptom_2']).
↳fillna(0))
    df['weighted_sym'] = (df['symptom_1'].fillna(0)*1 + df['symptom_2'].
↳fillna(0)*1 +
        df['symptom_3'].fillna(0)*1 + df['symptom_4'].
↳fillna(0)*2 +
        df['symptom_5'].fillna(0)*2)
    df['both_parents_defect'] = ((df['mother_defect'] == 1) &
↳(df['father_defect'] == 1)).astype(int)
    df['no_parent_defect'] = ((df['mother_defect'] == 0) &
↳(df['father_defect'] == 0)).astype(int)

    df = df.fillna(-1)
    return df

X_train_raw = preprocess(TRAIN_DATA)
X_test_raw = preprocess(TEST_DATA)
y_train = TRAIN_LABEL['disorder'].values

# Same adversarial drops as best run
DROP_ADV = ['blood_cell_count', 'white_blood_cell_count', 'mother_age']
X_train = X_train_raw.drop(columns=DROP_ADV)
X_test = X_test_raw.drop(columns=DROP_ADV)

print(f'X_train: {X_train.shape}, X_test: {X_test.shape}')

```

X_train: (13249, 56), X_test: (8834, 56)

```

[5]: # Cell 5: Class weights + params
class_counts = np.bincount(y_train)
class_weights = len(y_train) / (10 * class_counts)

# KEY CHANGE vs best run:
# learning_rate: 0.03 → 0.01 (3x slower learning)
# iterations: 2000 → 5000 (more trees to compensate)
# early_stopping: 100 → 300 (give it more patience at lower lr)
#
# WHY: many folds in 9-seed run stopped at iter 1-5.
# With lr=0.03 + depth=8, the model learns too fast - the first
# few trees capture everything and then immediately overfits.
# Lower lr forces smaller steps, more trees, smoother generalisation.

PARAMS = {
    'iterations' : 5000,
    'learning_rate' : 0.01, # ← KEY CHANGE (was 0.03)
    'depth' : 8,
    'l2_leaf_reg' : 1.2554074561515052,
    'random_strength' : 0.15300699009014024,
    'rsm' : 0.6693037260566354,
    'bagging_temperature' : 0.7561917100518147,
    'min_data_in_leaf' : 23,
    'early_stopping_rounds' : 300, # ← more patience at lower lr
    'eval_metric' : 'Accuracy',
    'verbose' : 0,
    'thread_count' : -1,
}

print('Params:')
for k, v in PARAMS.items():
    print(f' {k}: {v}')

```

```

Params:
iterations: 5000
learning_rate: 0.01
depth: 8
l2_leaf_reg: 1.2554074561515052
random_strength: 0.15300699009014024
rsm: 0.6693037260566354
bagging_temperature: 0.7561917100518147
min_data_in_leaf: 23
early_stopping_rounds: 300
eval_metric: Accuracy
verbose: 0
thread_count: -1

```

```

[6]: # Cell 6: Train - 6 seeds x 10 folds
print(f'Training {len(SEEDS)} seeds x {N_SPLITS} folds')
print('Note: will take ~2x longer than best run due to lower lr + more iters')

all_oof_proba = np.zeros((len(y_train), 10))
all_test_preds = np.zeros((len(X_test), 10))
seed_oof_scores = []

for SEED in SEEDS:
    print(f"\n{'='*40} SEED={SEED} {'='*40}")
    skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)
    oof_proba = np.zeros((len(y_train), 10))
    test_preds = np.zeros((len(X_test), 10))
    fold_scores = []

    for fold, (tr_idx, val_idx) in enumerate(skf.split(X_train, y_train)):
        X_tr, X_val = X_train.iloc[tr_idx], X_train.iloc[val_idx]
        y_tr, y_val = y_train[tr_idx], y_train[val_idx]

        model = CatBoostClassifier(
            class_weights = class_weights,
            random_seed = SEED,
            **PARAMS
        )
        model.fit(X_tr, y_tr, eval_set=(X_val, y_val), use_best_model=True)

        val_proba = model.predict_proba(X_val)
        score = balanced_accuracy_score(y_val, np.argmax(val_proba, axis=1))
        fold_scores.append(score)
        print(f' Fold {fold+1:2d}: BA={score:.4f} best_iter={model.
↪best_iteration_}')
        oof_proba[val_idx] += val_proba
        test_preds += model.predict_proba(X_test) / N_SPLITS

    oof_score = balanced_accuracy_score(y_train, np.argmax(oof_proba, axis=1))
    seed_oof_scores.append(oof_score)
    print(f' OOF BA (seed={SEED}): {oof_score:.4f} | mean={np.
↪mean(fold_scores):.4f} ± {np.std(fold_scores):.4f}')
    all_oof_proba += oof_proba / len(SEEDS)
    all_test_preds += test_preds / len(SEEDS)

final_oof = balanced_accuracy_score(y_train, np.argmax(all_oof_proba, axis=1))
print(f"\n{'='*60}")
print(f'Per-seed OOF: {[round(s,4) for s in seed_oof_scores]}')
print(f'Seed std: {np.std(seed_oof_scores):.4f}')
print(f'FINAL OOF BA (lr=0.01): {final_oof:.4f}')
print(f'Best run OOF (lr=0.03): 0.3949 (LB 0.37468)')

```

```

print(f'Change:                {final_oof - 0.3949:+.4f}')
```

```

print(f'Predicted LB   OOF - 0.020: {final_oof - 0.020:.4f}')
```

```

print(f"'{' '*60}")
```

Training 6 seeds x 10 folds

Note: will take ~2x longer than best run due to lower lr + more iters

===== SEED=42

=====

```

Fold  1: BA=0.3774  best_iter=127
Fold  2: BA=0.4048  best_iter=12
Fold  3: BA=0.4229  best_iter=9
Fold  4: BA=0.3952  best_iter=9
Fold  5: BA=0.4085  best_iter=240
Fold  6: BA=0.4434  best_iter=256
Fold  7: BA=0.3602  best_iter=2
Fold  8: BA=0.3776  best_iter=20
Fold  9: BA=0.4232  best_iter=159
Fold 10: BA=0.4540  best_iter=35
OOF BA (seed=42): 0.4067 | mean=0.4067 ± 0.0285
```

===== SEED=7

=====

```

Fold  1: BA=0.4100  best_iter=200
Fold  2: BA=0.4193  best_iter=17
Fold  3: BA=0.4103  best_iter=241
Fold  4: BA=0.4153  best_iter=217
Fold  5: BA=0.3539  best_iter=149
Fold  6: BA=0.4217  best_iter=243
Fold  7: BA=0.3714  best_iter=227
Fold  8: BA=0.3667  best_iter=12
Fold  9: BA=0.4196  best_iter=5
Fold 10: BA=0.3908  best_iter=198
OOF BA (seed=7): 0.3981 | mean=0.3979 ± 0.0240
```

===== SEED=123

=====

```

Fold  1: BA=0.4152  best_iter=269
Fold  2: BA=0.3782  best_iter=10
Fold  3: BA=0.4019  best_iter=11
Fold  4: BA=0.3925  best_iter=2
Fold  5: BA=0.4517  best_iter=195
Fold  6: BA=0.4186  best_iter=198
Fold  7: BA=0.3884  best_iter=210
Fold  8: BA=0.3794  best_iter=128
Fold  9: BA=0.3930  best_iter=105
Fold 10: BA=0.3930  best_iter=29
OOF BA (seed=123): 0.4017 | mean=0.4012 ± 0.0210
```

===== SEED=13

=====

Fold 1: BA=0.3847 best_iter=302
Fold 2: BA=0.4072 best_iter=311
Fold 3: BA=0.4023 best_iter=1
Fold 4: BA=0.4045 best_iter=263
Fold 5: BA=0.3892 best_iter=315
Fold 6: BA=0.4444 best_iter=273
Fold 7: BA=0.4306 best_iter=142
Fold 8: BA=0.4109 best_iter=188
Fold 9: BA=0.3824 best_iter=297
Fold 10: BA=0.3870 best_iter=231
OOF BA (seed=13): 0.4056 | mean=0.4043 $\pm$ 0.0194

===== SEED=99

=====

Fold 1: BA=0.3798 best_iter=16
Fold 2: BA=0.3765 best_iter=15
Fold 3: BA=0.4013 best_iter=280
Fold 4: BA=0.4489 best_iter=408
Fold 5: BA=0.3763 best_iter=4
Fold 6: BA=0.4580 best_iter=316
Fold 7: BA=0.3861 best_iter=18
Fold 8: BA=0.4038 best_iter=52
Fold 9: BA=0.4056 best_iter=166
Fold 10: BA=0.3997 best_iter=17
OOF BA (seed=99): 0.4035 | mean=0.4036 $\pm$ 0.0272

===== SEED=2024

=====

Fold 1: BA=0.4106 best_iter=139
Fold 2: BA=0.4202 best_iter=241
Fold 3: BA=0.3599 best_iter=274
Fold 4: BA=0.4490 best_iter=298
Fold 5: BA=0.4165 best_iter=7
Fold 6: BA=0.4044 best_iter=5
Fold 7: BA=0.3917 best_iter=32
Fold 8: BA=0.4182 best_iter=238
Fold 9: BA=0.3905 best_iter=270
Fold 10: BA=0.3961 best_iter=131
OOF BA (seed=2024): 0.4059 | mean=0.4057 $\pm$ 0.0224

=====

Per-seed OOF: [0.4067, 0.3981, 0.4017, 0.4056, 0.4035, 0.4059]

Seed std: 0.0029

FINAL OOF BA (lr=0.01): 0.3945

Best run OOF (lr=0.03): 0.3949 (LB 0.37468)

Change: -0.0004
 Predicted LB OOF - 0.020: 0.3745

=====

```
[7]: # Cell 7: Per-class recall
disorder_names = {
    0: ' ', 1: ' ', 2: ' ', 3: ' ', 4: ' ',
    5: ' - ', 6: ' ', 7: ' ', 8: ' ', 9: ' '
}
best_recall = {0:0.411, 1:0.362, 2:0.296, 3:0.319, 4:0.759,
               5:0.309, 6:0.503, 7:0.253, 8:0.560, 9:0.175}

oof_labels = np.argmax(all_oof_proba, axis=1)
report = classification_report(y_train, oof_labels, output_dict=True)

print(f'OOF BA: {final_oof:.4f} (best: 0.3949 → LB 0.37468)\n')
print(f'{"Class":<5} {"Name":<16} {"6-seed best":>12} {"lr=0.01":>8} {"Δ":>7}')
print('-' * 55)
for cls in range(10):
    r = report[str(cls)]['recall']
    r_old = best_recall[cls]
    delta = r - r_old
    flag = ' ← up' if delta > 0.02 else ' ← LOW' if r < 0.3 else ''
    print(f'{cls:<5} {disorder_names[cls]:<16} {r_old:>12.3f} {r:>8.3f} {delta:
    ↪>+7.3f}{flag}')
```

OOF BA: 0.3945 (best: 0.3949 → LB 0.37468)

Class Name	6-seed best	lr=0.01	Δ
0	0.411	0.416	+0.005
1	0.362	0.369	+0.007
2	0.296	0.303	+0.007
3	0.319	0.318	-0.001
4	0.759	0.741	-0.018
5 -	0.309	0.306	-0.003
6	0.503	0.513	+0.010
7	0.253	0.249	-0.004 ← LOW
8	0.560	0.560	+0.000
9	0.175	0.170	-0.005 ← LOW

```
[8]: # Cell 8: Save submission
final_preds = np.argmax(all_test_preds, axis=1)
submission = pd.DataFrame({
    'id': TEST_DATA.index,
    'disorder': final_preds
}).set_index('id')
submission.to_csv('submission_lr001.csv')
```



```

print('Saved: submission_lr001.csv')
print(f'Shape: {submission.shape}')
print('\nPrediction distribution:')
print(submission['disorder'].value_counts().sort_index())
print(f'\nFinal OOF: {final_oof:.4f}')
print(f'Predicted LB: {final_oof - 0.020:.4f}')
print(f'Current best LB: 0.37468')
print(f'\nSubmit ONLY if:')
print(f'  1. Predicted LB > 0.37468')
print(f'  2. Seed std < 0.005')
print(f'  3. best_iter values are NOT mostly 1-5 (check Cell 6)')

```

Saved: submission_lr001.csv
 Shape: (8834, 1)

Prediction distribution:
 disorder

0	481
1	1262
2	705
3	1460
4	231
5	1176
6	1069
7	1194
8	187
9	1069

Name: count, dtype: int64

Final OOF: 0.3945

Predicted LB: 0.3745

Current best LB: 0.37468

Submit ONLY if:

1. Predicted LB > 0.37468
2. Seed std < 0.005
3. best_iter values are NOT mostly 1-5 (check Cell 6)